Part IV

Systems of Equations

Lecture 20. Gaussian Elimination

Gaussian elimination is undoubtedly familiar to the reader. It is the simplest way to solve linear systems of equations by hand, and also the standard method for solving them on computers. We first describe Gaussian elimination in its pure form, and then, in the next lecture, add the feature of row pivoting that is essential to stability.

LU Factorization

Gaussian elimination transforms a full linear system into an upper-triangular one by applying simple linear transformations on the left. In this respect it is analogous to Householder triangularization for computing QR factorizations. The difference is that the transformations applied in Gaussian elimination are not unitary.

Let $A \in \mathbb{C}^{m\times m}$ be a square matrix. (The algorithm can also be applied to rectangular matrices, but as this is rarely done in practice, we shall confine our attention to the square case.) The idea is to transform A into an $m \times m$ upper-triangular matrix U by introducing zeros below the diagonal, first in column 1, then in column 2, and so on—just as in Householder triangularization. This is done by subtracting multiples of each row from subsequent rows. This "elimination" process is equivalent to multiplying A by a sequence of lower-triangular matrices L_k on the left:

$$\underbrace{L_{m-1}\cdots L_2 L_1}_{L^{-1}} A \;=\; U. \tag{20.1}$$

Setting $L = L_1^{-1} L_2^{-1} \cdots L_{m-1}^{-1}$ gives $A = LU$. Thus we obtain an *LU factorization* of A,

$$A = LU, \tag{20.2}$$

where U is upper-triangular and L is lower-triangular. It turns out that L is *unit lower-triangular*, which means that all of its diagonal entries are equal to 1.

For example, suppose we start with a 4×4 matrix. The algorithm proceeds in three steps (compare (10.1)):

$$\underset{A}{\begin{bmatrix} \times & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \end{bmatrix}} \xrightarrow{L_1} \underset{L_1 A}{\begin{bmatrix} \times & \times & \times & \times \\ \mathbf{0} & \mathbf{\times} & \mathbf{\times} & \mathbf{\times} \\ \mathbf{0} & \mathbf{\times} & \mathbf{\times} & \mathbf{\times} \\ \mathbf{0} & \mathbf{\times} & \mathbf{\times} & \mathbf{\times} \end{bmatrix}} \xrightarrow{L_2} \underset{L_2 L_1 A}{\begin{bmatrix} \times & \times & \times & \times \\ & \times & \times & \times \\ & \mathbf{0} & \mathbf{\times} & \mathbf{\times} \\ & \mathbf{0} & \mathbf{\times} & \mathbf{\times} \end{bmatrix}} \xrightarrow{L_3} \underset{L_3 L_2 L_1 A}{\begin{bmatrix} \times & \times & \times & \times \\ & \times & \times & \times \\ & & \times & \times \\ & & \mathbf{0} & \mathbf{\times} \end{bmatrix}}.$$

(As in Lecture 10, boldfacing indicates entries just operated upon, and blank entries are zero.) The kth transformation L_k introduces zeros below the diagonal in column k by subtracting multiples of row k from rows $k+1, \ldots, m$. Since the first $k-1$ entries of row k are already zero, this operation does not destroy any zeros previously introduced.

Gaussian elimination thus augments our taxonomy of algorithms for factoring a matrix:

Gram–Schmidt: $A = QR$ by triangular orthogonalization,
Householder: $A = QR$ by orthogonal triangularization,
Gaussian elimination: $A = LU$ by triangular triangularization.

Example

In discussing the details, it will help to have a numerical example on the table. Suppose we start with the 4×4 matrix

$$A = \begin{bmatrix} 2 & 1 & 1 & 0 \\ 4 & 3 & 3 & 1 \\ 8 & 7 & 9 & 5 \\ 6 & 7 & 9 & 8 \end{bmatrix}. \tag{20.3}$$

(The entries of A are anything but random; they were chosen to give a simple LU factorization.) The first step of Gaussian elimination looks like this:

$$L_1 A = \begin{bmatrix} 1 & & & \\ -2 & 1 & & \\ -4 & & 1 & \\ -3 & & & 1 \end{bmatrix} \begin{bmatrix} 2 & 1 & 1 & 0 \\ 4 & 3 & 3 & 1 \\ 8 & 7 & 9 & 5 \\ 6 & 7 & 9 & 8 \end{bmatrix} = \begin{bmatrix} 2 & 1 & 1 & 0 \\ & 1 & 1 & 1 \\ & 3 & 5 & 5 \\ & 4 & 6 & 8 \end{bmatrix}.$$

In words, we have subtracted twice the first row from the second, four times the first row from the third, and three times the first row from the fourth. The second step looks like this:

$$L_2L_1A = \begin{bmatrix} 1 & & & \\ & 1 & & \\ -3 & 1 & & \\ -4 & & 1 & \end{bmatrix} \begin{bmatrix} 2 & 1 & 1 & 0 \\ & 1 & 1 & 1 \\ & 3 & 5 & 5 \\ & 4 & 6 & 8 \end{bmatrix} = \begin{bmatrix} 2 & 1 & 1 & 0 \\ & 1 & 1 & 1 \\ & & 2 & 2 \\ & & 2 & 4 \end{bmatrix}.$$

This time we have subtracted three times the second row from the third and four times the second row from the fourth. Finally, in the third step we subtract the third row from the fourth:

$$L_3L_2L_1A = \begin{bmatrix} 1 & & & \\ & 1 & & \\ & & 1 & \\ & & -1 & 1 \end{bmatrix} \begin{bmatrix} 2 & 1 & 1 & 0 \\ & 1 & 1 & 1 \\ & & 2 & 2 \\ & & 2 & 4 \end{bmatrix} = \begin{bmatrix} 2 & 1 & 1 & 0 \\ & 1 & 1 & 1 \\ & & 2 & 2 \\ & & & 2 \end{bmatrix} = U.$$

Now, to exhibit the full factorization $A = LU$, we need to compute the product $L = L_1^{-1}L_2^{-1}L_3^{-1}$. Perhaps surprisingly, this turns out to be a triviality. The inverse of L_1 is just L_1 itself, but with each entry below the diagonal negated:

$$\begin{bmatrix} 1 & & & \\ -2 & 1 & & \\ -4 & & 1 & \\ -3 & & & 1 \end{bmatrix}^{-1} = \begin{bmatrix} 1 & & & \\ 2 & 1 & & \\ 4 & & 1 & \\ 3 & & & 1 \end{bmatrix}. \tag{20.4}$$

Similarly, the inverses of L_2 and L_3 are obtained by negating their subdiagonal entries. Finally, the product $L_1^{-1}L_2^{-1}L_3^{-1}$ is just the unit lower-triangular matrix with the nonzero subdiagonal entries of L_1^{-1}, L_2^{-1}, and L_3^{-1} inserted in the appropriate places. All together, we have

$$\underset{A}{\begin{bmatrix} 2 & 1 & 1 & 0 \\ 4 & 3 & 3 & 1 \\ 8 & 7 & 9 & 5 \\ 6 & 7 & 9 & 8 \end{bmatrix}} = \underset{L}{\begin{bmatrix} 1 & & & \\ 2 & 1 & & \\ 4 & 3 & 1 & \\ 3 & 4 & 1 & 1 \end{bmatrix}} \underset{U}{\begin{bmatrix} 2 & 1 & 1 & 0 \\ & 1 & 1 & 1 \\ & & 2 & 2 \\ & & & 2 \end{bmatrix}}. \tag{20.5}$$

General Formulas and Two Strokes of Luck

Here are the general formulas for an $m \times m$ matrix. Suppose x_k denotes the kth column of the matrix at the beginning of step k. Then the transformation

L_k must be chosen so that

$$x_k = \begin{bmatrix} x_{1k} \\ \vdots \\ x_{kk} \\ x_{k+1,k} \\ \vdots \\ x_{mk} \end{bmatrix} \quad \begin{matrix} L_k \\ \longrightarrow \end{matrix} \quad L_k x_k = \begin{bmatrix} x_{1k} \\ \vdots \\ x_{kk} \\ 0 \\ \vdots \\ 0 \end{bmatrix}.$$

To do this we wish to subtract ℓ_{jk} times row k from row j, where ℓ_{jk} is the *multiplier*

$$\ell_{jk} = \frac{x_{jk}}{x_{kk}} \qquad (k < j \le m). \tag{20.6}$$

The matrix L_k takes the form

$$L_k = \begin{bmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & & -\ell_{k+1,k} & 1 & \\ & & \vdots & & \ddots \\ & & -\ell_{mk} & & 1 \end{bmatrix},$$

with the nonzero subdiagonal entries situated in column k. This is analogous to (10.2) for Householder triangularization.

In the numerical example above, we noted two strokes of luck: that L_k can be inverted by negating its subdiagonal entries (20.4), and that L can be formed by collecting the entries ℓ_{jk} in the appropriate places (20.5). We can explain these bits of good fortune as follows. Let us define

$$\ell_k = \begin{bmatrix} 0 \\ \vdots \\ 0 \\ \ell_{k+1,k} \\ \vdots \\ \ell_{m,k} \end{bmatrix}.$$

Then L_k can be written $L_k = I - \ell_k e_k^*$, where e_k is, as usual, the column vector with 1 in position k and 0 elsewhere. The sparsity pattern of ℓ_k implies $e_k^* \ell_k = 0$, and therefore $(I - \ell_k e_k^*)(I + \ell_k e_k^*) = I - \ell_k e_k^* \ell_k e_k^* = I$. In other words, the inverse of L_k is $I + \ell_k e_k^*$, as in (20.4).

For the second stroke of luck we argue as follows. Consider, for example, the product $L_k^{-1} L_{k+1}^{-1}$. From the sparsity pattern of ℓ_{k+1}, we have $e_k^* \ell_{k+1} = 0$, and therefore

$$L_k^{-1} L_{k+1}^{-1} = (I + \ell_k e_k^*)(I + \ell_{k+1} e_{k+1}^*) = I + \ell_k e_k^* + \ell_{k+1} e_{k+1}^*.$$

Thus $L_k^{-1}L_{k+1}^{-1}$ is just the unit lower-triangular matrix with the entries of both L_k^{-1} and L_{k+1}^{-1} inserted in their usual places below the diagonal. When we take the product of all of these matrices to form L, we have the same convenient property everywhere below the diagonal:

$$L = L_1^{-1}L_2^{-1}\cdots L_{m-1}^{-1} = \begin{bmatrix} 1 & & & & \\ \ell_{21} & 1 & & & \\ \ell_{31} & \ell_{32} & 1 & & \\ \vdots & \vdots & \ddots & \ddots & \\ \ell_{m1} & \ell_{m2} & \cdots & \ell_{m,m-1} & 1 \end{bmatrix}. \tag{20.7}$$

Though we did not mention it in Lecture 8, the sparsity considerations that led to (20.7) also appeared in the interpretation (8.10) of the modified Gram–Schmidt process as a succession of right-multiplications by triangular matrices R_k.

In practical Gaussian elimination, the matrices L_k are never formed and multiplied explicitly. The multipliers ℓ_{jk} are computed and stored directly into L, and the transformations L_k are then applied implicitly.

Algorithm 20.1. Gaussian Elimination without Pivoting

$U = A$, $L = I$
for $k = 1$ **to** $m-1$
    for $j = k+1$ **to** m
        $\ell_{jk} = u_{jk}/u_{kk}$
        $u_{j,k:m} = u_{j,k:m} - \ell_{jk}u_{k,k:m}$

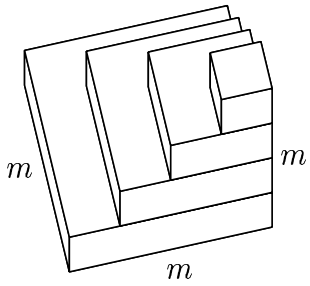
(Three matrices A, L, U are not really needed; to minimize memory use on the computer, both L and U can be written into the same array as A.) See Exercise 20.4 for an alternative "outer product" formulation of Gaussian elimination, involving one explicit loop rather than two.

Operation Count

As usual, the asymptotic operation count of this algorithm can be derived geometrically. The work is dominated by the vector operation in the inner loop, $u_{j,k:m} = u_{j,k:m} - \ell_{jk}u_{k,k:m}$, which executes one scalar-vector multiplication and one vector subtraction. If $l = m - k + 1$ denotes the length of the row vectors being manipulated, the number of flops is $2l$: two flops per entry.

For each value of k, the inner loop is repeated for rows $k+1, \ldots, m$. The

work involved corresponds to one layer of the following solid:



This is the same figure we displayed in Lecture 10 to represent the work done in Householder triangularization (assuming $m = n$). There, however, each unit cube represented four flops rather than two. As before, the solid converges as $m \to \infty$ to a pyramid, with volume $\frac{1}{3}m^3$. At two flops per unit of volume, this adds up to

$$\text{Work for Gaussian elimination:} \quad \sim \frac{2}{3}m^3 \text{ flops.} \tag{20.8}$$

Solution of $Ax = b$ by LU Factorization

If A is factored into L and U, a system of equations $Ax = b$ is reduced to the form $LUx = b$. Thus it can be solved by solving two triangular systems: first $Ly = b$ for the unknown y (forward substitution), then $Ux = y$ for the unknown x (back substitution). The first step requires $\sim \frac{2}{3}m^3$ flops, and the second and third each require $\sim m^2$ flops. The total work is $\sim \frac{2}{3}m^3$ flops, half the figure of $\sim \frac{4}{3}m^3$ flops (10.9) for a solution by Householder triangularization (Algorithm 16.1).

Why is Gaussian elimination usually used rather than QR factorization to solve square systems of equations? The factor of 2 is certainly one reason. Also important, however, may be the historical fact that the elimination idea has been known for centuries, whereas QR factorization of matrices did not come along until after the invention of computers. To supplant Gaussian elimination as the method of choice, QR factorization would have to have had a compelling advantage.

Instability of Gaussian Elimination without Pivoting

Unfortunately, Gaussian elimination as presented so far is unusable for solving general linear systems, for it is not backward stable. The instability is related to another, more obvious difficulty. For certain matrices, Gaussian elimination fails entirely, because it attempts division by zero.

For example, consider

$$A = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}.$$

This matrix has full rank and is well-conditioned, with $\kappa(A) = (3+\sqrt{5})/2 \approx 2.618$ in the 2-norm. Nevertheless, Gaussian elimination fails at the first step.

A slight perturbation of the same matrix reveals the more general problem. Suppose we apply Gaussian elimination to

$$A = \begin{bmatrix} 10^{-20} & 1 \\ 1 & 1 \end{bmatrix}. \tag{20.9}$$

Now the process does not fail. Instead, 10^{20} times the first row is subtracted from the second row, and the following factors are produced:

$$L = \begin{bmatrix} 1 & 0 \\ 10^{20} & 1 \end{bmatrix}, \qquad U = \begin{bmatrix} 10^{-20} & 1 \\ 0 & 1-10^{20} \end{bmatrix}.$$

However, suppose these computations are performed in floating point arithmetic with $\epsilon_{\text{machine}} \approx 10^{-16}$. The number $1-10^{20}$ will not be represented exactly; it will be rounded to the nearest floating point number. For simplicity, imagine that this is exactly -10^{20}. Then the floating point matrices produced by the algorithm will be

$$\tilde{L} = \begin{bmatrix} 1 & 0 \\ 10^{20} & 1 \end{bmatrix}, \qquad \tilde{U} = \begin{bmatrix} 10^{-20} & 1 \\ 0 & -10^{20} \end{bmatrix}.$$

This degree of rounding might seem tolerable at first. After all, the matrix $\tilde{U}$ is close to the correct U relative to $\|U\|$. However, the problem becomes apparent when we compute the product $\tilde{L}\tilde{U}$:

$$\tilde{L}\tilde{U} = \begin{bmatrix} 10^{-20} & 1 \\ 1 & 0 \end{bmatrix}.$$

This matrix is not at all close to A, for the 1 in the $(2,2)$ position has been replaced by 0. If we now solve the system $\tilde{L}\tilde{U}x = b$, the result will be nothing like the solution to $Ax = b$. For example, with $b = (1,0)^*$ we get $\tilde{x} = (0,1)^*$, whereas the correct solution is $x \approx (-1,1)^*$.

A careful consideration of what has occurred in this example reveals the following. Gaussian elimination has computed the LU factorization stably: $\tilde{L}$ and $\tilde{U}$ are close to the exact factors for a matrix close to A (in fact, A itself). Yet it has not solved $Ax = b$ stably. The explanation is that the LU factorization, though stable, was *not backward stable*. As a rule, if one step of an algorithm is a stable but not backward stable algorithm for solving a subproblem, the stability of the overall calculation may be in jeopardy.

In fact, for general $m \times m$ matrices A, the situation is worse than this. Gaussian elimination without pivoting is neither backward stable nor stable as a general algorithm for LU factorization. Additionally, the triangular matrices it generates have condition numbers that may be arbitrarily greater than those of A itself, leading to additional sources of instability in the forward and back substitution phases of the solution of $Ax = b$.

Exercises

20.1. Let $A \in \mathbb{C}^{m\times m}$ be nonsingular. Show that A has an LU factorization if and only if for each k with $1 \le k \le m$, the upper-left $k \times k$ block $A_{1:k,1:k}$ is nonsingular. (Hint: The row operations of Gaussian elimination leave the determinants $\det(A_{1:k,1:k})$ unchanged.) Prove that this LU factorization is unique.

20.2. Suppose $A \in \mathbb{C}^{m\times m}$ satisfies the condition of Exercise 20.1 and is banded with bandwidth $2p+1$, i.e., $a_{ij} = 0$ for $|i-j| > p$. What can you say about the sparsity patterns of the factors L and U of A?

20.3. Suppose an $m\times m$ matrix A is written in the block form $A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}$, where A_{11} is $n \times n$ and A_{22} is $(m-n) \times (m-n)$.
Assume that A satisfies the condition of Exercise 20.1.

(a) Verify the formula

$$\begin{bmatrix} I & \\ -A_{21}A_{11}^{-1} & I \end{bmatrix} \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ & A_{22} - A_{21}A_{11}^{-1}A_{12} \end{bmatrix}$$

for "elimination" of the block A_{21}. The matrix $A_{22} - A_{21}A_{11}^{-1}A_{12}$ is known as the *Schur complement* of A_{11} in A.

(b) Suppose A_{21} is eliminated row by row by means of n steps of Gaussian elimination. Show that the bottom-right $(m-n) \times (m-n)$ block of the result is again $A_{22} - A_{21}A_{11}^{-1}A_{12}$.

20.4. Like most of the algorithms in this book, Gaussian elimination involves a triply nested loop. In Algorithm 20.1, there are two explicit **for** loops, and the third loop is implicit in the vectors $u_{j,k:m}$ and $u_{k,k:m}$. Rewrite this algorithm with just one explicit **for** loop indexed by k. Inside this loop, U will be updated at each step by a certain rank-one outer product. This "outer product" form of Gaussian elimination may be a better starting point than Algorithm 20.1 if one wants to optimize computer performance.

20.5. We have seen that Gaussian elimination yields a factorization $A = LU$, where L has ones on the diagonal but U does not. Describe at a high level the factorization that results if this process is varied in the following ways:

(a) Elimination by columns from left to right, rather than by rows from top to bottom, so that A is made lower-triangular.

(b) Gaussian elimination applied after a preliminary scaling of the columns of A by a diagonal matrix D. What form does a system $Ax = b$ take under this rescaling? Is it the equations or the unknowns that are rescaled by D?

(c) Gaussian elimination carried further, so that after A (assumed nonsingular) is brought to upper-triangular form, additional column operations are carried out so that this upper-triangular matrix is made diagonal.

Lecture 21. Pivoting

In the last lecture we saw that Gaussian elimination in its pure form is unstable. The instability can be controlled by permuting the order of the rows of the matrix being operated on, an operation called *pivoting*. Pivoting has been a standard feature of Gaussian elimination computations since the 1950s.

Pivots

At step k of Gaussian elimination, multiples of row k are subtracted from rows $k+1, \ldots, m$ of the working matrix X in order to introduce zeros in entry k of these rows. In this operation row k, column k, and especially the entry x_{kk} play special roles. We call x_{kk} the *pivot*. From every entry in the submatrix $X_{k+1:m,k:m}$ is subtracted the product of a number in row k and a number in column k, divided by x_{kk}:

$$\begin{bmatrix} \times & \times & \times & \times & \times \\ & x_{kk} & \times & \times & \times \\ & \times & \times & \times & \times \\ & \times & \times & \times & \times \\ & \times & \times & \times & \times \end{bmatrix} \longrightarrow \begin{bmatrix} \times & \times & \times & \times & \times \\ & x_{kk} & \times & \times & \times \\ & \mathbf{0} & \times & \times & \times \\ & \mathbf{0} & \times & \times & \times \\ & \mathbf{0} & \times & \times & \times \end{bmatrix}.$$

However, there is no reason why the kth row and column must be chosen for the elimination. For example, we could just as easily introduce zeros in column k by adding multiples of some row i with $k < i \leq m$ to the other rows

$k, \ldots, m$. In this case, the entry x_{ik} would be the pivot. Here is an illustration with $k = 2$ and $i = 4$:

$$\begin{bmatrix} \times & \times & \times & \times & \times \\ & \times & \times & \times & \times \\ & \times & \times & \times & \times \\ & x_{ik} & \times & \times & \times \\ & \times & \times & \times & \times \end{bmatrix} \longrightarrow \begin{bmatrix} \times & \times & \times & \times & \times \\ & \mathbf{0} & \times & \times & \times \\ & \mathbf{0} & \times & \times & \times \\ & x_{ik} & \times & \times & \times \\ & \mathbf{0} & \times & \times & \times \end{bmatrix}.$$

Similarly, we could introduce zeros in column j rather than column k. Here is an illustration with $k = 2$, $i = 4$, $j = 3$:

$$\begin{bmatrix} \times & \times & \times & \times & \times \\ & \times & \times & \times & \times \\ & \times & \times & \times & \times \\ & \times & x_{ij} & \times & \times \\ & \times & \times & \times & \times \end{bmatrix} \longrightarrow \begin{bmatrix} \times & \times & \times & \times & \times \\ & \times & \mathbf{0} & \times & \times \\ & \times & \mathbf{0} & \times & \times \\ & \times & x_{ij} & \times & \times \\ & \times & \mathbf{0} & \times & \times \end{bmatrix}.$$

All in all, we are free to choose any entry of $X_{k:m,k:m}$ as the pivot, as long as it is nonzero. The possibility that an entry $x_{kk} = 0$ might arise implies that some flexibility of choice of the pivot may sometimes be necessary, even from a pure mathematical point of view. For numerical stability, however, it is desirable to pivot even when x_{kk} is nonzero if there is a larger element available. In practice, it is common to pick as pivot the largest number among a set of entries being considered as candidates.

The structure of the elimination process quickly becomes confusing if zeros are introduced in arbitrary patterns through the matrix. To see what is going on, we want to retain the triangular structure described in the last lecture, and there is an easy way to do this. We shall not think of the pivot x_{ij} as left in place, as in the illustrations above. Instead, at step k, we shall imagine that the rows and columns of the working matrix are permuted so as to move x_{ij} into the (k, k) position. Then, when the elimination is done, zeros are introduced into entries $k+1, \ldots, m$ of column k, just as in Gaussian elimination without pivoting. This interchange of rows and perhaps columns is what is usually thought of as *pivoting*.

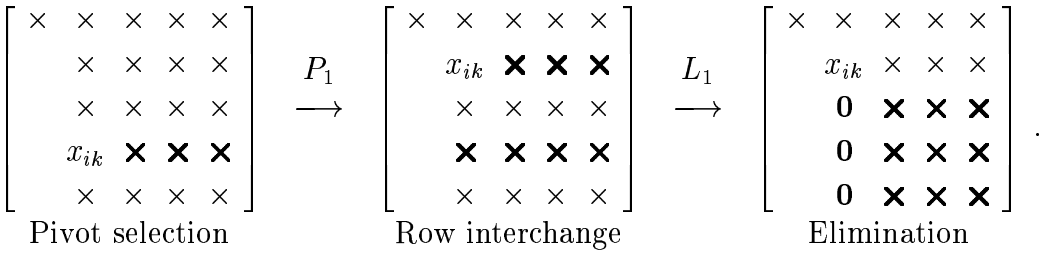
The idea that rows and columns are interchanged is indispensable conceptually. Whether it is a good idea to interchange them physically on the computer is less clear. In some implementations, the data in computer memory are indeed swapped at each pivot step. In others, an equivalent effect is achieved by indirect addressing with permuted index vectors. Which approach is best varies from machine to machine and depends on many factors.

Partial Pivoting

If every entry of $X_{k:m,k:m}$ is considered as a possible pivot at step k, there are $O((m-k)^2)$ entries to be examined to determine the largest. Summing over

m steps, the total cost of selecting pivots becomes $O(m^3)$ operations, adding significantly to the cost of Gaussian elimination, not to mention the potential difficulties of global communication in an unpredictable pattern across all the entries of a matrix. This expensive strategy is called *complete pivoting*.

In practice, equally good pivots can be found by considering a much smaller number of entries. The standard method for doing this is *partial pivoting*. Here, only rows are interchanged. The pivot at each step is chosen as the largest of the $m-k+1$ subdiagonal entries in column k, incurring a total cost of only $O(m-k)$ operations for selecting the pivot at each step, hence $O(m^2)$ operations overall. To bring the kth pivot into the (k,k) position, no columns need to be permuted; it is enough to swap row k with the row containing the pivot.



As usual in numerical linear algebra, this algorithm can be expressed as a matrix product. We saw in the last lecture that an elimination step corresponds to left-multiplication by an elementary lower-triangular matrix L_k. Partial pivoting complicates matters by applying a permutation matrix P_k on the left of the working matrix before each elimination. (A permutation matrix is a matrix with 0 everywhere except for a single 1 in each row and column. That is, it is a matrix obtained from the identity by permuting rows or columns.) After $m-1$ steps, A becomes an upper-triangular matrix U:

$$L_{m-1}P_{m-1}\cdots L_2P_2L_1P_1A \;=\; U. \tag{21.1}$$

Example

To see what is going on, it will be helpful to return to the numerical example (20.3) of the last lecture,

$$A \;=\; \begin{bmatrix} 2 & 1 & 1 & 0 \\ 4 & 3 & 3 & 1 \\ 8 & 7 & 9 & 5 \\ 6 & 7 & 9 & 8 \end{bmatrix}. \tag{21.2}$$

With partial pivoting, the first thing we do is interchange the first and third rows (left-multiplication by P_1):

$$\begin{bmatrix} & & 1 & \\ & 1 & & \\ 1 & & & \\ & & & 1 \end{bmatrix} \begin{bmatrix} 2 & 1 & 1 & 0 \\ 4 & 3 & 3 & 1 \\ 8 & 7 & 9 & 5 \\ 6 & 7 & 9 & 8 \end{bmatrix} = \begin{bmatrix} 8 & 7 & 9 & 5 \\ 4 & 3 & 3 & 1 \\ 2 & 1 & 1 & 0 \\ 6 & 7 & 9 & 8 \end{bmatrix}.$$

The first elimination step now looks like this (left-multiplication by L_1):

$$\begin{bmatrix} 1 & & & \\ -\frac{1}{2} & 1 & & \\ -\frac{1}{4} & & 1 & \\ -\frac{3}{4} & & & 1 \end{bmatrix} \begin{bmatrix} 8 & 7 & 9 & 5 \\ 4 & 3 & 3 & 1 \\ 2 & 1 & 1 & 0 \\ 6 & 7 & 9 & 8 \end{bmatrix} = \begin{bmatrix} 8 & 7 & 9 & 5 \\ & -\frac{1}{2} & -\frac{3}{2} & -\frac{3}{2} \\ & -\frac{3}{4} & -\frac{5}{4} & -\frac{5}{4} \\ & \frac{7}{4} & \frac{9}{4} & \frac{17}{4} \end{bmatrix}.$$

Now the second and fourth rows are interchanged (multiplication by P_2):

$$\begin{bmatrix} 1 & & & \\ & & & 1 \\ & & 1 & \\ & 1 & & \end{bmatrix} \begin{bmatrix} 8 & 7 & 9 & 5 \\ & -\frac{1}{2} & -\frac{3}{2} & -\frac{3}{2} \\ & -\frac{3}{4} & -\frac{5}{4} & -\frac{5}{4} \\ & \frac{7}{4} & \frac{9}{4} & \frac{17}{4} \end{bmatrix} = \begin{bmatrix} 8 & 7 & 9 & 5 \\ & \frac{7}{4} & \frac{9}{4} & \frac{17}{4} \\ & -\frac{3}{4} & -\frac{5}{4} & -\frac{5}{4} \\ & -\frac{1}{2} & -\frac{3}{2} & -\frac{3}{2} \end{bmatrix}.$$

The second elimination step then looks like this (multiplication by L_1):

$$\begin{bmatrix} 1 & & & \\ & 1 & & \\ \frac{3}{7} & 1 & & \\ \frac{2}{7} & & 1 & \end{bmatrix} \begin{bmatrix} 8 & 7 & 9 & 5 \\ & \frac{7}{4} & \frac{9}{4} & \frac{17}{4} \\ & -\frac{3}{4} & -\frac{5}{4} & -\frac{5}{4} \\ & -\frac{1}{2} & -\frac{3}{2} & -\frac{3}{2} \end{bmatrix} = \begin{bmatrix} 8 & 7 & 9 & 5 \\ & \frac{7}{4} & \frac{9}{4} & \frac{17}{4} \\ & & -\frac{2}{7} & \frac{4}{7} \\ & & -\frac{6}{7} & -\frac{2}{7} \end{bmatrix}.$$

Now the third and fourth rows are interchanged (multiplication by P_3):

$$\begin{bmatrix} 1 & & & \\ & 1 & & \\ & & & 1 \\ & & 1 & \end{bmatrix} \begin{bmatrix} 8 & 7 & 9 & 5 \\ & \frac{7}{4} & \frac{9}{4} & \frac{17}{4} \\ & & -\frac{2}{7} & \frac{4}{7} \\ & & -\frac{6}{7} & -\frac{2}{7} \end{bmatrix} = \begin{bmatrix} 8 & 7 & 9 & 5 \\ & \frac{7}{4} & \frac{9}{4} & \frac{17}{4} \\ & & -\frac{6}{7} & -\frac{2}{7} \\ & & -\frac{2}{7} & \frac{4}{7} \end{bmatrix}.$$

The final elimination step looks like this (multiplication by L_3):

$$\begin{bmatrix} 1 & & & \\ & 1 & & \\ & & 1 & \\ & -\frac{1}{3} & 1 & \end{bmatrix} \begin{bmatrix} 8 & 7 & 9 & 5 \\ & \frac{7}{4} & \frac{9}{4} & \frac{17}{4} \\ & & -\frac{6}{7} & -\frac{2}{7} \\ & & -\frac{2}{7} & \frac{4}{7} \end{bmatrix} = \begin{bmatrix} 8 & 7 & 9 & 5 \\ & \frac{7}{4} & \frac{9}{4} & \frac{17}{4} \\ & & -\frac{6}{7} & -\frac{2}{7} \\ & & & \frac{2}{3} \end{bmatrix}.$$

$PA = LU$ Factorization and a Third Stroke of Luck

Have we just computed an LU factorization of A? Not quite, but almost. In fact, we have computed an LU factorization of PA, where P is a permutation matrix. It looks like this:

$$\underset{P}{\begin{bmatrix} & 1 & & \\ & & & 1 \\ 1 & & & \\ & & 1 & \end{bmatrix}} \underset{A}{\begin{bmatrix} 2 & 1 & 1 & 0 \\ 4 & 3 & 3 & 1 \\ 8 & 7 & 9 & 5 \\ 6 & 7 & 9 & 8 \end{bmatrix}} = \underset{L}{\begin{bmatrix} 1 & & & \\ \frac{3}{4} & 1 & & \\ \frac{1}{2} & -\frac{2}{7} & 1 & \\ \frac{1}{4} & -\frac{3}{7} & \frac{1}{3} & 1 \end{bmatrix}} \underset{U}{\begin{bmatrix} 8 & 7 & 9 & 5 \\ & \frac{7}{4} & \frac{9}{4} & \frac{17}{4} \\ & & -\frac{6}{7} & -\frac{2}{7} \\ & & & \frac{2}{3} \end{bmatrix}}. \tag{21.3}$$

This formula should be compared with (20.5). The presence of integers there and fractions here is not a general distinction, but an artifact of our choice of A. The distinction that matters is that here, all the subdiagonal entries of L are ≤ 1 in magnitude, a consequence of the property $|x_{kk}| = \max_j |x_{jk}|$ in (20.6) introduced by pivoting.

It is not obvious where (21.3) comes from. Our elimination process took the form

$$L_3 P_3 L_2 P_2 L_1 P_1 A = U,$$

which doesn't look lower-triangular at all. But here, a third stroke of good fortune has come to our aid. These six elementary operations can be reordered in the form

$$L_3 P_3 L_2 P_2 L_1 P_1 = L_3' L_2' L_1' P_3 P_2 P_1, \tag{21.4}$$

where L_k' is equal to L_k but with the subdiagonal entries permuted. To be precise, define

$$L_3' = L_3, \quad L_2' = P_3 L_2 P_3^{-1}, \quad L_1' = P_3 P_2 L_1 P_2^{-1} P_3^{-1}.$$

Since each of these definitions applies only permutations P_j with $j > k$ to L_k, it is easily verified that L_k' has the same structure as L_k. Computing the product of the matrices L_k' reveals

$$L_3' L_2' L_1' P_3 P_2 P_1 = L_3 (P_3 L_2 P_3^{-1})(P_3 P_2 L_1 P_2^{-1} P_3^{-1}) P_3 P_2 P_1 = L_3 P_3 L_2 P_2 L_1 P_1,$$

as in (21.4).

In general, for an $m \times m$ matrix, the factorization (21.1) provided by Gaussian elimination with partial pivoting can be written in the form

$$(L_{m-1}' \cdots L_2' L_1')(P_{m-1} \cdots P_2 P_1) A = U, \tag{21.5}$$

where L_k' is defined by

$$L_k' = P_{m-1} \cdots P_{k+1} L_k P_{k+1}^{-1} \cdots P_{m-1}^{-1}. \tag{21.6}$$

The product of the matrices L_k' is unit lower-triangular and easily invertible by negating the subdiagonal entries, just as in Gaussian elimination without pivoting. Writing $L = (L_{m-1}' \cdots L_2' L_1')^{-1}$ and $P = P_{m-1} \cdots P_2 P_1$, we have

$$PA = LU. \tag{21.7}$$

In general, any square matrix A, singular or nonsingular, has a factorization (21.7), where P is a permutation matrix, L is unit lower-triangular with lower-triangular entries ≤ 1 in magnitude, and U is upper-triangular. Partial pivoting is such a universal practice that this factorization is usually known simply as an *LU factorization* of A.

The famous formula (21.7) has a simple interpretation. Gaussian elimination with partial pivoting is equivalent to the following procedure:

1. Permute the rows of A according to P
2. Apply Gaussian elimination without pivoting to PA.

Partial pivoting is not carried out this way in practice, of course, since P is not known ahead of time.

Here is a formal statement of the algorithm.

Algorithm 21.1. Gaussian Elimination with Partial Pivoting

$U = A,\ L = I,\ P = I$
for $k = 1$ **to** $m-1$
    Select $i \ge k$ to maximize $|u_{ik}|$
    $u_{k,k:m} \leftrightarrow u_{i,k:m}$ (interchange two rows)
    $\ell_{k,1:k-1} \leftrightarrow \ell_{i,1:k-1}$
    $p_{k,:} \leftrightarrow p_{i,:}$
    for $j = k+1$ **to** m
        $\ell_{jk} = u_{jk}/u_{kk}$
        $u_{j,k:m} = u_{j,k:m} - \ell_{jk} u_{k,k:m}$

To leading order, this algorithm requires the same number of floating point operations (20.8) as Gaussian elimination without pivoting, namely, $\frac{2}{3}m^3$. As with Algorithm 20.1, the use of computer memory can be minimized if desired by overwriting U and L into the same array used to store A.

In practice, of course, P is not represented explicitly as a matrix. The rows are swapped at each step, or an equivalent effect is achieved via a permutation vector, as indicated earlier.

Complete Pivoting

In complete pivoting, the selection of pivots takes a significant amount of time. In practice this is rarely done, because the improvement in stability is marginal. However, we shall outline how the algebra changes in this case.

In matrix form, complete pivoting precedes each elimination step with a permutation P_k of the rows applied on the left and also a permutation Q_k of the columns applied on the right:

$$L_{m-1}P_{m-1}\cdots L_2P_2L_1P_1AQ_1Q_2\cdots Q_{m-1} \;=\; U. \tag{21.8}$$

Once again, this is not quite an LU factorization of A, but it is close. If the L_k' are defined as in (21.6) (the column permutations are not involved), then

$$(L_{m-1}'\cdots L_2'L_1')(P_{m-1}\cdots P_2P_1)A(Q_1Q_2\cdots Q_{m-1}) = U. \tag{21.9}$$

Setting $L = (L_{m-1}'\cdots L_2'L_1')^{-1}$, $P = P_{m-1}\cdots P_2P_1$, and $Q = Q_1Q_2\cdots Q_{m-1}$, we obtain

$$PAQ = LU. \tag{21.10}$$

Exercises

21.1. Let A be the 4×4 matrix (20.3) considered in this lecture and the previous one.

(a) Determine $\det A$ from (20.5).

(b) Determine $\det A$ from (21.3).

(c) Describe how Gaussian elimination with partial pivoting can be used to find the determinant of a general square matrix.

21.2. Suppose $A \in \mathbb{C}^{m\times m}$ is banded with bandwidth $2p+1$, as in Exercise 20.2, and a factorization $PA = LU$ is computed by Gaussian elimination with partial pivoting. What can you say about the sparsity patterns of L and U?

21.3. Consider Gaussian elimination carried out with pivoting by columns instead of rows, leading to a factorization $AQ = LU$, where Q is a permutation matrix.

(a) Show that if A is nonsingular, such a factorization always exists.

(b) Show that if A is singular, such a factorization does not always exist.

21.4. Gaussian elimination can be used to compute the inverse A^{-1} of a nonsingular matrix $A \in \mathbb{C}^{m\times m}$, though it is rarely really necessary to do so.

(a) Describe an algorithm for computing A^{-1} by solving m systems of equations, and show that its asymptotic operation count is $8m^3/3$ flops.

(b) Describe a variant of your algorithm, taking advantage of sparsity, that reduces the operation count to $2m^3$ flops.

(c) Suppose one wishes to solve n systems of equations $Ax_j = b_j$, or equivalently, a block system $AX = B$ with $B \in \mathbb{C}^{m\times n}$. What is the asymptotic operation count (a function of m and n) for doing this (i) directly from the LU factorization and (ii) with a preliminary computation of A^{-1}?

21.5. Suppose $A \in \mathbb{C}^{m\times m}$ is hermitian, or in the real case, symmetric (but not necessarily positive definite).

(a) Describe a strategy of *symmetric pivoting* to preserve the hermitian structure while still leading to a unit lower-triangular matrix with entries $|\ell_{ij}| \le 1$.

(b) What is the form of the matrix factorization computed by your algorithm?

(c) What is its asymptotic operation count?

21.6. Suppose $A \in \mathbb{C}^{m\times m}$ is *strictly column diagonally dominant*, which means that for each k,

$$|a_{kk}| > \sum_{j\ne k} |a_{jk}|. \tag{21.11}$$

Show that if Gaussian elimination with partial pivoting is applied to A, no row interchanges take place.

21.7. In Lecture 20 the "two strokes of luck" were explained by the use of the vectors e_k and ℓ_k. Give an explanation based on these vectors for the "third stroke of luck" in the present lecture.

Lecture 22. Stability of Gaussian Elimination

Gaussian elimination with partial pivoting is explosively unstable for certain matrices, yet stable in practice. This apparent paradox has a statistical explanation.

Stability and the Size of L and U

The stability analysis of most algorithms of numerical linear algebra, including virtually all of those based on unitary operations, is straightforward. The stability analysis of Gaussian elimination with partial pivoting, however, is complicated, and has been a point of difficulty in numerical analysis since the 1950s. This is one of the reasons why we saved Gaussian elimination until the second half of this book.

In (20.9) we gave an example of a 2×2 matrix for which Gaussian elimination without pivoting was unstable. In that example, the factor L had an entry of size 10^{20}. An attempt to solve a system of equations based on L introduced rounding errors of relative order $\epsilon_{\text{machine}}$, hence absolute order $\epsilon_{\text{machine}} \times 10^{20}$. Not surprisingly, this destroyed the accuracy of the result.

It turns out that this example is, in a sense, entirely general. Instability in Gaussian elimination—with or without pivoting—can arise only if one or both of the factors L and U is large relative to the size of A. Thus the purpose of pivoting, from the point of view of stability, is to ensure that L and U are not too large. As long as all the intermediate quantities that arise during the

elimination are of manageable size, the rounding errors they emit are very small, and the algorithm is backward stable.

The following theorem makes this idea precise. It is stated for Gaussian elimination without pivoting, but it applies to elimination with pivoting too if A is taken to represent the original matrix with appropriately permuted rows and/or columns.

Theorem 22.1. *Let the factorization $A = LU$ of a nonsingular matrix $A \in \mathbb{C}^{m\times m}$ be computed by Gaussian elimination without pivoting (Algorithm 20.1) on a computer satisfying the axioms (13.5) and (13.7). If A has an LU factorization, then for all sufficiently small $\epsilon_{\text{machine}}$, the factorization completes successfully in floating point arithmetic (no zero pivots are encountered), and the computed matrices $\tilde{L}$ and $\tilde{U}$ satisfy*

$$\tilde{L}\tilde{U} = A + \delta A, \qquad \frac{\|\delta A\|}{\|L\|\,\|U\|} = O(\epsilon_{\text{machine}}) \tag{22.1}$$

for some $\delta A \in \mathbb{C}^{m\times n}$.

As usual in numerical linear algebra, we make no claims about $\tilde{L}-L$ or $\tilde{U}-U$, only about $\tilde{L}\tilde{U} - LU$.

At first glance this estimate may look like half a dozen others in this book, such as (16.3) or (17.3). What makes it different is that the quantity in the denominator is $\|L\|\,\|U\|$, not $\|A\|$. If $\|L\|\,\|U\| = O(\|A\|)$, then (22.1) asserts that Gaussian elimination is backward stable. If $\|L\|\,\|U\| \neq O(\|A\|)$, however, we must expect backward instability.

For Gaussian elimination without pivoting, both L and U can be unboundedly large. That algorithm is unstable by any standard, and we shall not discuss it further. Instead, from now on, we shall confine our attention to Gaussian elimination with partial pivoting.

Growth Factors

Consider Gaussian elimination with partial pivoting. Because each pivot selection involves maximization over a column, this algorithm produces a matrix L with entries of absolute value ≤ 1 everywhere below the diagonal. This implies $\|L\| = O(1)$ in any norm. Therefore, for Gaussian elimination with partial pivoting, (22.1) reduces to the condition $\|\delta A\|/\|U\| = O(\epsilon_{\text{machine}})$. We conclude that the algorithm is backward stable provided $\|U\| = O(\|A\|)$.

There is a standard reformulation of this conclusion that is perhaps more vivid. Gaussian elimination reduces a full matrix A to an upper-triangular matrix U. We have just seen that the key question for stability is whether amplification of the entries takes place during this reduction. In particular, let the *growth factor* for A be defined as the ratio

$$\rho \;=\; \frac{\max_{i,j}|u_{ij}|}{\max_{i,j}|a_{ij}|}. \tag{22.2}$$

If ρ is of order 1, not much growth has taken place, and the elimination process is stable. If ρ is bigger than this, we must expect instability. Specifically, since $\|L\| = O(1)$, and since (22.2) implies $\|U\| = O(\rho\|A\|)$, the following result is a corollary of Theorem 22.1.

Theorem 22.2. *Let the factorization $PA = LU$ of a matrix $A \in \mathbb{C}^{m\times m}$ be computed by Gaussian elimination with partial pivoting (Algorithm 21.1) on a computer satisfying the axioms (13.5) and (13.7). Then the computed matrices $\tilde{P}$, $\tilde{L}$, and $\tilde{U}$ satisfy*

$$\tilde{L}\tilde{U} = \tilde{P}A + \delta A, \qquad \frac{\|\delta A\|}{\|A\|} = O(\rho\,\epsilon_{\text{machine}}) \tag{22.3}$$

for some $\delta A \in \mathbb{C}^{m\times n}$, where ρ is the growth factor for A. If $|\ell_{ij}| < 1$ for each $i > j$, implying that there are no ties in the selection of pivots in exact arithmetic, then $\tilde{P} = P$ for all sufficiently small $\epsilon_{\text{machine}}$.

Is Gaussian elimination backward stable? According to Theorem 22.2 and our definition (14.5) of backward stability, the answer is yes if $\rho = O(1)$ uniformly for all matrices of a given dimension m, and otherwise no.

And now, the complications begin.

Worst-Case Instability

For certain matrices A, despite the beneficial effects of pivoting, ρ turns out to be huge. For example, suppose A is the matrix

$$A = \begin{bmatrix} 1 & & & & 1 \\ -1 & 1 & & & 1 \\ -1 & -1 & 1 & & 1 \\ -1 & -1 & -1 & 1 & 1 \\ -1 & -1 & -1 & -1 & 1 \end{bmatrix}. \tag{22.4}$$

At the first step, no pivoting takes place, but entries $2, 3, \ldots, m$ in the final column are doubled from 1 to 2. Another doubling occurs at each subsequent elimination step. At the end we have

$$U = \begin{bmatrix} 1 & & & & 1 \\ & 1 & & & 2 \\ & & 1 & & 4 \\ & & & 1 & 8 \\ & & & & 16 \end{bmatrix}. \tag{22.5}$$

The final $PA = LU$ factorization looks like this:

$$\begin{bmatrix} 1 & & & & 1 \\ -1 & 1 & & & 1 \\ -1 & -1 & 1 & & 1 \\ -1 & -1 & -1 & 1 & 1 \\ -1 & -1 & -1 & -1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & & & & \\ -1 & 1 & & & \\ -1 & -1 & 1 & & \\ -1 & -1 & -1 & 1 & \\ -1 & -1 & -1 & -1 & 1 \end{bmatrix} \begin{bmatrix} 1 & & & & 1 \\ & 1 & & & 2 \\ & & 1 & & 4 \\ & & & 1 & 8 \\ & & & & 16 \end{bmatrix}.$$

For this 5×5 matrix, the growth factor is $\rho = 16$. For an $m \times m$ matrix of the same form, it is $\rho = 2^{m-1}$. (This is as large as ρ can get; see Exercise 22.1.)

A growth factor of order 2^m corresponds to a loss of on the order of m bits of precision, which is catastrophic for a practical computation. Since a typical computer represents floating point numbers with just sixty-four bits, whereas matrix problems of dimensions in the hundreds or thousands are solved all the time, a loss of m bits of precision is intolerable for real computations.

This brings us to an awkward point. Here, in the discussion of Gaussian elimination with pivoting—for the only time in this book—the definitions of stability presented in Lecture 14 fail us. According to the definitions, all that matters in determining stability or backward stability is the existence of a certain bound applicable uniformly to all matrices *for each fixed dimension* m. Uniformity with respect to m is not required. Here, for each m, we have a uniform bound involving the constant 2^{m-1}. Thus, according to our definitions, Gaussian elimination is backward stable.

Theorem 22.3. *According to the definitions of Lecture* 14*, Gaussian elimination with partial pivoting is backward stable.*

This conclusion is absurd, however, in view of the vastness of 2^{m-1} for practical values of m.

For the remainder of this lecture, we ask the reader to put aside our formal definitions of stability and accept a more informal (and more standard) use of words. Gaussian elimination for certain matrices is explosively unstable, as can be confirmed by numerical experiments with MATLAB, LINPACK, LAPACK, or other software packages of impeccable reputation (Exercise 22.2).

Stability in Practice

If Gaussian elimination is unstable, why is it so famous and so popular? This brings us to a point that is not just an artifact of definitions but a fundamental fact about the behavior of this algorithm. *Despite examples like* (22.4)*, Gaussian elimination with partial pivoting is utterly stable in practice. Large factors* U *like* (22.5) *never seem to appear in real applications. In fifty years of computing, no matrix problems that excite an explosive instability are known to have arisen under natural circumstances.*

This is a curious situation indeed. How can an algorithm that fails for certain matrices be entirely trustworthy in practice? The answer seems to be that although some matrices cause instability, these represent such an extraordinarily small proportion of the set of all matrices that they "never" arise in practice simply for statistical reasons.

One can learn more about this phenomenon by considering random matrices. Of course, the matrices that arise in applications are not random in any ordinary sense. They have all kinds of special properties, and if one tried to describe them as random samples from some distribution, it would have to be a curious distribution indeed. It would certainly be unreasonable to expect that any particular distribution of random matrices should match the behavior of the matrices arising in practice in a close quantitative way.

However, the phenomenon to be explained is not a matter of precise quantities. Matrices with large growth factors are vanishingly rare in applications. If we can show that they are vanishingly rare among random matrices in some well-defined class, the mechanisms involved must surely be the same. The argument does not depend on one measure of "vanishingly" agreeing with the other to any particular factor such as 2 or 10 or 100.

Figures 22.1 and 22.2 present experiments with random matrices as defined in Exercise 12.3: each entry is an independent sample from the real normal distribution of mean 0 and standard deviation $m^{-1/2}$. In Figure 22.1, a collection of random matrices of various dimensions have been factored and the growth factors presented as a scatter plot. Only two of the matrices gave a growth factor as large as $m^{1/2}$. In Figure 22.2, the results of factoring one million matrices each of dimensions $m = 8$, 16, and 32 are shown. Here, the growth factors have been collected in bins of width 0.2 and the resulting data plotted as a probability density distribution. The probability density of growth factors appears to decrease exponentially with size. Among these three million matrices, though the maximum growth factor in principle might have been 2,147,483,648, the maximum actually encountered was 11.99.

Similar results are obtained with random matrices defined by other probability distributions, such as uniformly distributed entries in $[-1, 1]$ (Exercise 22.3). If you pick a billion matrices at random, you will almost certainly not find one for which Gaussian elimination is unstable.

Explanation

We shall not attempt to give a full explanation of why the matrices for which Gaussian elimination is unstable are so rare. This would not be possible, as the matter is not yet fully understood. But we shall present an outline of an explanation.

If $PA = LU$, then $U = L^{-1}PA$. It follows that if Gaussian elimination is unstable when applied to the matrix A, implying that ρ is large, then L^{-1} must be large too. Now, as it happens, random triangular matrices tend

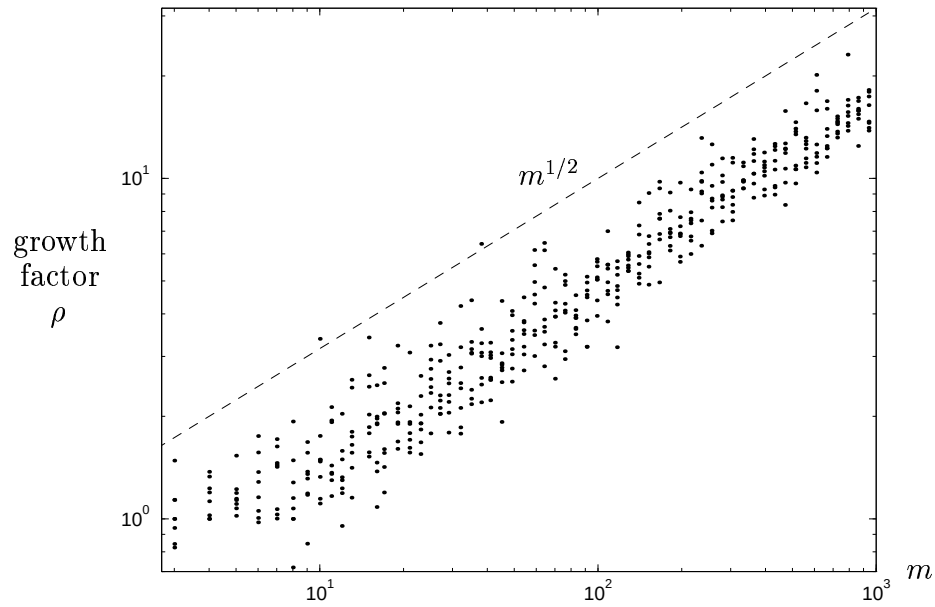


Figure 22.1. *Growth factors for Gaussian elimination with partial pivoting applied to* 496 *random matrices (independent, normally distributed entries) of various dimensions. The typical size of* ρ *is of order* $m^{1/2}$*, much less than the maximal possible value* 2^{m-1}*.*

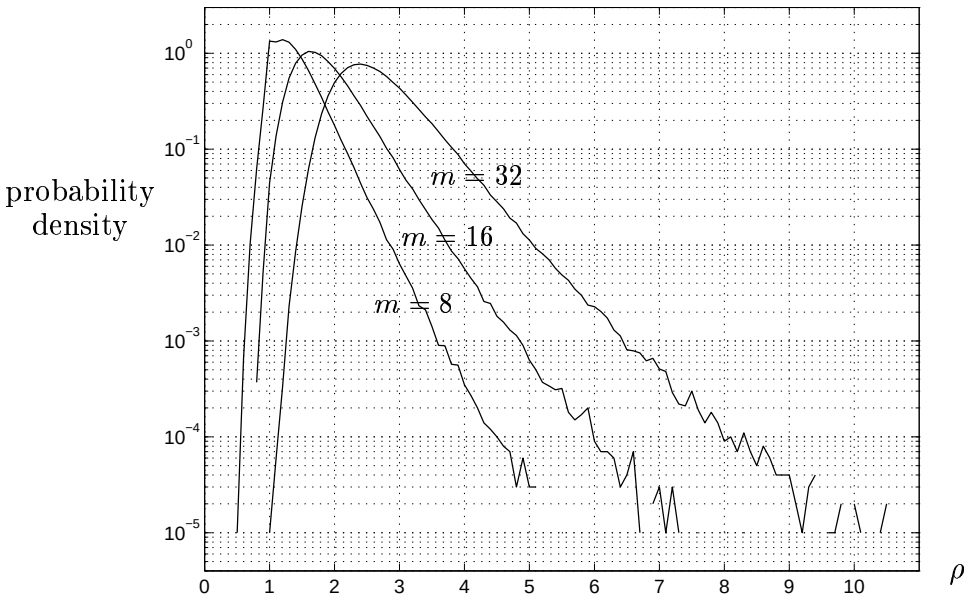


Figure 22.2. *Probability density distributions for growth factors of random matrices of dimensions* $m = 8$, 16, 32*, based on sample sizes of one million for each dimension. The density appears to decrease exponentially with* ρ*. The chatter near the end of each curve is an artifact of the finite sample sizes.*

to have huge inverses, exponentially large as a function of the dimension m (Exercise 12.3(d)). In particular, this is true for random triangular matrices of the form delivered by Gaussian elimination with partial pivoting, with 1 on the diagonal and entries ≤ 1 in absolute value below.

When Gaussian elimination is applied to random matrices A, however, the resulting factors L are anything but random. Correlations appear among the signs of the entries of L that render these matrices extraordinarily well-conditioned. A typical entry of L^{-1}, far from being exponentially large, is usually less than 1 in absolute value. Figure 22.3 presents evidence of this phenomenon based on a single (but typical) matrix of dimension $m = 128$.

We thus arrive at the question: why do the matrices L delivered by Gaussian elimination almost never have large inverses?

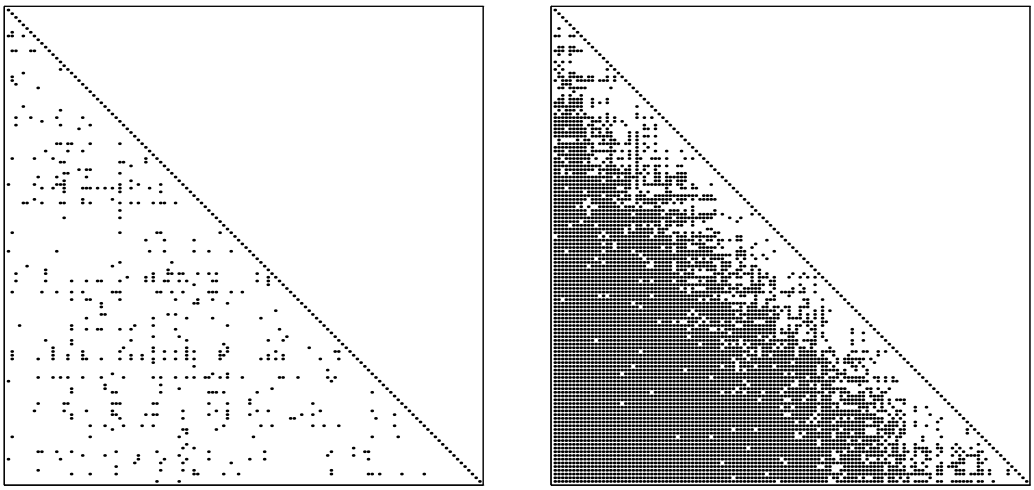
The answer lies in the consideration of column spaces. Since U is upper-triangular and $PA = LU$, the column spaces of PA and L are the same. By this we mean that the first column of PA spans the same space as the first column of L, the first two columns of PA span the same space as the first two columns of L, and so on. If A is random, its column spaces are randomly oriented, and it follows that the same must be true of the column spaces of $P^{-1}L$. However, this condition is incompatible with L^{-1} being large. It can be shown that if L^{-1} is large, then the column spaces of L, or of any permutation $P^{-1}L$, must be skewed in a fashion that is very far from random.

Figure 22.4 gives evidence of this. The figure shows "where the energy is" in the successive column spaces of the same two matrices as in Figure 22.3. The device for doing this is a *Q portrait*, defined by the MATLAB commands

```
[Q,R] = qr(A),     spy( abs(Q) > 1/sqrt(m) ).        (22.6)
```

These commands first compute a QR factorization of the matrix A, then plot a dot at each position of Q corresponding to an entry larger than the standard deviation, $m^{-1/2}$. The figure illustrates that for a random A, even after row interchanges to the form PA, the column spaces are oriented nearly randomly, whereas for a matrix A that gives a large growth factor, the orientations are very far from random. It is likely that by quantifying this argument, it can be proved that growth factors larger than order $m^{1/2}$ are exponentially rare among random matrices in the sense that for any $\alpha > 1/2$ and $M > 0$, the probability of the event $\rho > m^{\alpha}$ is smaller than m^{-M} for all sufficiently large m. As of this writing, however, such a theorem has not yet been proved.

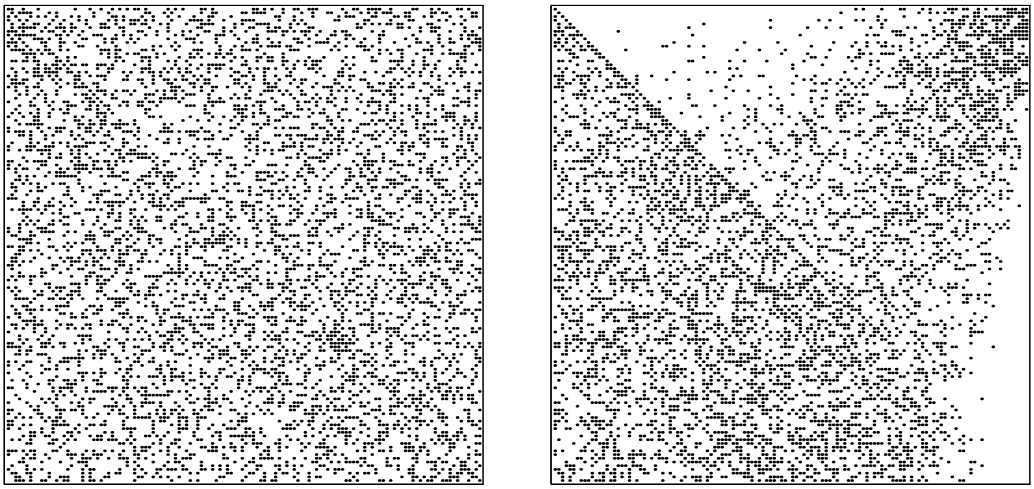
Let us summarize the stability of Gaussian elimination with partial pivoting. This algorithm is highly unstable for certain matrices A. For instability to occur, however, the column spaces of A must be skewed in a very special fashion, one that is exponentially rare in at least one class of random matrices. Decades of computational experience suggest that matrices whose column spaces are skewed in this fashion arise very rarely in applications.



random A
$\max_{i,j} |(L^{-1})_{ij}| = 2.67$

random $\tilde{L}$
$\max_{i,j} |(\tilde{L}^{-1})_{ij}| = 2.27 \times 10^4$

Figure 22.3. *Let A be a random 128×128 matrix with factorization $PA = LU$. On the left, L^{-1} is shown: the dots represent entries with magnitude ≥ 1. On the right, a similar picture for $\tilde{L}^{-1}$, where $\tilde{L}$ is the same as L except that the signs of its subdiagonal entries have been randomized. Gaussian elimination tends to produce matrices L that are extraordinarily well-conditioned.*



random A

random $\tilde{L}$

Figure 22.4. *Q portraits* (22.6) *of the same two matrices. On the left, the random matrix A after permutation to the form PA, or equivalently, the factor L. On the right, the matrix $\tilde{L}$ with randomized signs. The column spaces of $\tilde{L}$ are skewed in a manner exponentially unlikely to arise in typical classes of random matrices.*

Exercises

22.1. Show that for Gaussian elimination with partial pivoting applied to any matrix $A \in \mathbb{C}^{m \times m}$, the growth factor (22.2) satisfies $\rho \leq 2^{m-1}$.

22.2. Experiment with solving 60×60 systems of equations $Ax = b$ by Gaussian elimination with partial pivoting, with A having the form (22.4). Do you observe that the results are useless because of the growth factor of order 2^{60}? At your first attempt you may not observe this, because the integer entries of A may prevent any rounding errors from occurring. If so, find a way to modify your problem slightly so that the growth factor is the same or nearly so and catastrophic rounding errors really do take place.

22.3. Reproduce the figures of this lecture, approximately if not in full detail, but based on random matrices with entries uniformly distributed in $[-1, 1]$ rather than normally distributed. Do you see any significant differences?

22.4. (a) Suppose $PA = LU$ (LU factorization with partial pivoting) and $A = QR$ (QR factorization). Describe a relationship between the last row of L^{-1} and the last column of Q.

(b) Show that if A is random in the sense of having independent, normally distributed entries, then its column spaces are randomly oriented, so that in particular, the last column of Q is a random unit vector.

(c) Combine the results of (a) and (b) to make a statement about the final row of L^{-1} in Gaussian elimination applied to a random matrix A.

Lecture 23. Cholesky Factorization

Hermitian positive definite matrices can be decomposed into triangular factors twice as quickly as general matrices. The standard algorithm for this, Cholesky factorization, is a variant of Gaussian elimination that operates on both the left and the right of the matrix at once, preserving and exploiting symmetry.

Hermitian Positive Definite Matrices

A real matrix $A \in \mathbb{R}^{m\times m}$ is *symmetric* if it has the same entries below the diagonal as above: $a_{ij} = a_{ji}$ for all i, j, hence $A = A^T$. Such a matrix satisfies $x^T Ay = y^T Ax$ for all vectors $x, y \in \mathbb{R}^m$.

For a complex matrix $A \in \mathbb{C}^{m\times m}$, the analogous property is that A is *hermitian*. A hermitian matrix has entries below the diagonal that are complex conjugates of those above the diagonal: $a_{ij} = \overline{a_{ji}}$, hence $A = A^*$. (These definitions appeared already in Lecture 2.) Note that this means that the diagonal entries of a hermitian matrix must be real.

A hermitian matrix A satisfies $x^*Ay = \overline{y^*Ax}$ for all $x, y \in \mathbb{C}^m$. This means in particular that for any $x \in \mathbb{C}^m$, x^*Ax is real. If in addition $x^*Ax > 0$ for all $x \neq 0$, then A is said to be *hermitian positive definite* (or sometimes just *positive definite*). Many matrices that arise in physical systems are hermitian positive definite because of fundamental physical laws.

If A is an $m \times m$ hermitian positive definite matrix and X is an $m \times n$ matrix of full rank with $m \geq n$, then the matrix X^*AX is also hermitian positive definite. It is hermitian because $(X^*AX)^* = X^*A^*X = X^*AX$, and

it is positive definite because, for any vector $x \neq 0$, we have $Xx \neq 0$ and thus $x^*(X^*AX)x = (Xx)^*A(Xx) > 0$. By choosing X to be an $m \times n$ matrix with a 1 in each column and zeros elsewhere, we can write any $n \times n$ principal submatrix of A in the form X^*AX. Therefore, any principal submatrix of A must be positive definite. In particular, every diagonal entry of A is a positive real number.

The eigenvalues of a hermitian positive definite matrix are also positive real numbers. If $Ax = \lambda x$ for $x \neq 0$, we have $x^*Ax = \lambda x^*x > 0$ and therefore $\lambda > 0$. Conversely, it can be shown that if a hermitian matrix has all positive eigenvalues, then it is positive definite.

Eigenvectors that correspond to distinct eigenvalues of a hermitian matrix are orthogonal. (As discussed in the next lecture, hermitian matrices are *normal*.) Suppose $Ax_1 = \lambda_1 x_1$ and $Ax_2 = \lambda_2 x_2$ with $\lambda_1 \neq \lambda_2$. Then

$$\lambda_2 x_1^* x_2 = x_1^* A x_2 = \overline{x_2^* A x_1} = \overline{\lambda_1 x_2^* x_1} = \lambda_1 x_1^* x_2,$$

so $(\lambda_1 - \lambda_2) x_1^* x_2 = 0$. Since $\lambda_1 \neq \lambda_2$, we have $x_1^* x_2 = 0$.

Symmetric Gaussian Elimination

We turn now to the problem of decomposing a hermitian positive definite matrix into triangular factors. To begin, consider what happens if a single step of Gaussian elimination is applied to a hermitian matrix A with a 1 in the upper-left position:

$$A = \begin{bmatrix} 1 & w^* \\ w & K \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ w & I \end{bmatrix} \begin{bmatrix} 1 & w^* \\ 0 & K - ww^* \end{bmatrix}.$$

As described in Lecture 20, zeros have been introduced into the first column of the matrix by an elementary lower-triangular operation on the left that subtracts multiples of the first row from subsequent rows.

Gaussian elimination would now continue the reduction to triangular form by introducing zeros in the second column. However, in order to maintain symmetry, Cholesky factorization first introduces zeros in the first row to match the zeros just introduced in the first column. We can do this by a right upper-triangular operation that subtracts multiples of the first column from the subsequent ones:

$$\begin{bmatrix} 1 & w^* \\ 0 & K - ww^* \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & K - ww^* \end{bmatrix} \begin{bmatrix} 1 & w^* \\ 0 & I \end{bmatrix}.$$

Note that this upper-triangular operation is exactly the adjoint of the lower-triangular operation that we used to introduce zeros in the first column.

Combining the operations above, we find that the matrix A has been factored into three terms:

$$A = \begin{bmatrix} 1 & w^* \\ w & K \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ w & I \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & K - ww^* \end{bmatrix} \begin{bmatrix} 1 & w^* \\ 0 & I \end{bmatrix}. \tag{23.1}$$

The idea of Cholesky factorization is to continue this process, zeroing one column and one row of A symmetrically until it is reduced to the identity.

Cholesky Factorization

In order for the symmetric triangular reduction to work in general, we need a factorization that works for any $a_{11} > 0$, not just $a_{11} = 1$. The generalization of (23.1) is accomplished by adjusting some of the elements of R_1 by a factor of $\sqrt{a_{11}}$. Let $\alpha = \sqrt{a_{11}}$ and observe:

$$\begin{aligned} A &= \begin{bmatrix} a_{11} & w^* \\ w & K \end{bmatrix} \\ &= \begin{bmatrix} \alpha & 0 \\ w/\alpha & I \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & K - ww^*/a_{11} \end{bmatrix} \begin{bmatrix} \alpha & w^*/\alpha \\ 0 & I \end{bmatrix} = R_1^* A_1 R_1. \end{aligned}$$

This is the basic step that is applied repeatedly in Cholesky factorization. If the upper-left entry of the submatrix $K - ww^*/a_{11}$ is positive, the same formula can be used to factor it; we then have $A_1 = R_2^* A_2 R_2$ and thus $A = R_1^* R_2^* A_2 R_2 R_1$. The process is continued down to the bottom-right corner, giving us eventually a factorization

$$A = \underbrace{R_1^* R_2^* \cdots R_m^*}_{R^*} \underbrace{R_m \cdots R_2 R_1}_{R}. \tag{23.2}$$

This equation has the form

$$A = R^* R, \qquad r_{jj} > 0, \tag{23.3}$$

where R is upper-triangular. A reduction of this kind of a hermitian positive definite matrix is known as a *Cholesky factorization*.

The description above left one item dangling. How do we know that the upper-left entry of the submatrix $K - ww^*/a_{11}$ is positive? The answer is that it must be positive because $K - ww^*/a_{11}$ is positive definite, since it is the $(m-1)\times(m-1)$ lower-right principal submatrix of the positive definite matrix $R_1^{-*} A R_1^{-1}$. By induction, the same argument shows that all the submatrices A_j that appear in the course of the factorization are positive definite, and thus the process cannot break down. We can formalize this conclusion as follows.

Theorem 23.1. *Every hermitian positive definite matrix $A \in \mathbb{C}^{m\times m}$ has a unique Cholesky factorization* (23.3).

Proof. Existence is what we just discussed; a factorization exists since the algorithm cannot break down. In fact, the algorithm also establishes uniqueness. At each step (23.2), the value $\alpha = \sqrt{a_{11}}$ is determined by the form of

the R^*R factorization, and once α is determined, the first row of R_1^* is determined too. Since the analogous quantities are determined at each step of the reduction, the entire factorization is unique. □

The Algorithm

When Cholesky factorization is implemented, only half of the matrix being operated on needs to be represented explicitly. This simplification allows half of the arithmetic to be avoided. A formal statement of the algorithm (only one of many possibilities) is given below. The input matrix A represents the superdiagonal half of the $m \times m$ hermitian positive definite matrix to be factored. (In practical software, a compressed storage scheme may be used to avoid wasting half the entries of a square array.) The output matrix R represents the upper-triangular factor for which $A = R^*R$. Each outer iteration corresponds to a single elementary factorization: the upper-triangular part of the submatrix $R^*_{k:m,k:m}$ represents the superdiagonal part of the hermitian matrix being factored at step k.

Algorithm 23.1. Cholesky Factorization

$R = A$
for $k = 1$ **to** m
    for $j = k+1$ **to** m
        $R_{j,j:m} = R_{j,j:m} - R_{k,j:m}\overline{R}_{kj}/R_{kk}$
    $R_{k,k:m} = R_{k,k:m}/\sqrt{R_{kk}}$

Operation Count

The arithmetic done in Cholesky factorization is dominated by the inner loop. A single execution of the line

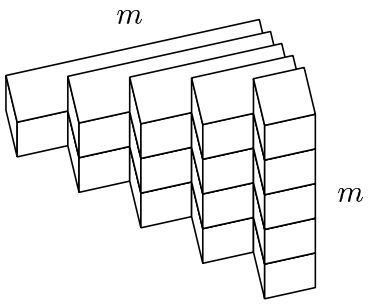
$$R_{j,j:m} = R_{j,j:m} - R_{k,j:m}\overline{R}_{kj}/R_{kk}$$

requires one division, $m - j + 1$ multiplications, and $m - j + 1$ subtractions, for a total of $\sim 2(m-j)$ flops. This calculation is repeated once for each j from $k+1$ to m, and that loop is repeated for each k from 1 to m. The sum is straightforward to evaluate:

$$\sum_{k=1}^{m}\sum_{j=k+1}^{m} 2(m-j) \;\sim\; 2\sum_{k=1}^{m}\sum_{j=1}^{k} j \;\sim\; \sum_{k=1}^{m} k^2 \;\sim\; \frac{1}{3}m^3 \text{ flops.}$$

Thus, Cholesky factorization involves only half as many operations as Gaussian elimination, which would require $\sim \frac{2}{3}m^3$ flops to factor the same matrix.

As usual, the operation count can also be determined graphically. For each k, two floating point operations are carried out (one multiplication and one subtraction) at each position of a triangular layer. The entire algorithm corresponds to stacking m layers:



As $m \to \infty$, the solid converges to a tetrahedron with volume $\frac{1}{6}m^3$. Since each unit cube corresponds to two floating point operations, we obtain again

$$\text{Work for Cholesky factorization:} \quad \sim \frac{1}{3}m^3 \text{ flops.} \tag{23.4}$$

Stability

All of the subtleties of the stability analysis of Gaussian elimination vanish for Cholesky factorization. This algorithm is always stable. Intuitively, the reason is that the factors R can never grow large. In the 2-norm, for example, we have $\|R\| = \|R^*\| = \|A\|^{1/2}$ (proof: SVD), and in other p-norms with $1 \le p \le \infty$, $\|R\|$ cannot differ from $\|A\|^{1/2}$ by more than a factor of $\sqrt{m}$. Thus, numbers much larger than the entries of A can never arise.

Note that the stability of Cholesky factorization is achieved without the need for any pivoting. Intuitively, one may observe that this is related to the fact that most of the weight of a hermitian positive definite matrix is on the diagonal. For example, it is not hard to show that the largest entry must appear on the diagonal, and this property carries over to the positive definite submatrices constructed in the inductive process (23.2).

An analysis of the stability of the Cholesky process leads to the following backward stability result.

Theorem 23.2. *Let $A \in \mathbb{C}^{m\times m}$ be hermitian positive definite, and let a Cholesky factorization of A be computed by Algorithm* 23.1 *on a computer satisfying* (13.5) *and* (13.7). *For all sufficiently small $\epsilon_{\text{machine}}$, this process is guaranteed to run to completion (i.e., no zero or negative corner entries r_{kk} will arise), generating a computed factor $\tilde{R}$ that satisfies*

$$\tilde{R}^*\tilde{R} = A + \delta A, \qquad \frac{\|\delta A\|}{\|A\|} = O(\epsilon_{\text{machine}}) \tag{23.5}$$

for some $\delta A \in \mathbb{C}^{m\times m}$.

Like so many algorithms of numerical linear algebra, this one would look much worse if we tried to carry out a forward error analysis rather than a backward one. If A is ill-conditioned, $\tilde{R}$ will not generally be close to R; the best we can say is $\|\tilde{R} - R\|/\|R\| = O(\kappa(A)\epsilon_{\text{machine}})$. (In other words, Cholesky factorization is in general an ill-conditioned problem.) It is only the product $\tilde{R}^*R$ that satisfies the much better error bound (23.5). Thus the errors introduced in $\tilde{R}$ by rounding are large but "diabolically correlated," just as we saw in Lecture 16 for QR factorization.

Solution of $Ax = b$

If A is hermitian positive definite, the standard way to solve a system of equations $Ax = b$ is by Cholesky factorization. Algorithm 23.1 reduces the system to $R^*Rx = b$, and we then solve two triangular systems in succession: first $R^*y = b$ for the unknown y, then $Rx = y$ for the unknown x. Each triangular solution requires just $\sim m^2$ flops, so the total work is again $\sim \frac{1}{3}m^3$ flops.

By reasoning analogous to that of Lecture 16, it can be shown that this process is backward stable.

Theorem 23.3. *The solution of hermitian positive definite systems $Ax = b$ via Cholesky factorization (Algorithm 23.1) is backward stable, generating a computed solution $\tilde{x}$ that satisfies*

$$(A + \Delta A)\tilde{x} = b, \qquad \frac{\|\Delta A\|}{\|A\|} = O(\epsilon_{\text{machine}}) \tag{23.6}$$

for some $\Delta A \in \mathbb{C}^{m\times m}$.

Exercises

23.1. Let A be a nonsingular square matrix and let $A = QR$ and $A^*A = U^*U$ be QR and Cholesky factorizations, respectively, with the usual normalizations r_{jj}, $u_{jj} > 0$. Is it true or false that $R = U$?

23.2. Using the proof of Theorem 16.2 as a guide, derive Theorem 23.3 from Theorems 23.2 and 17.1.

23.3. *Reverse Software Engineering of "\".* The following MATLAB session records a sequence of tests of the elapsed times for various computations on a workstation manufactured in 1991. For each part, try to explain: (i) Why was this experiment carried out? (ii) Why did the result come out as it did? Your

answers should refer to formulas from the text for flop counts. The MATLAB queries `help chol` and `help slash` may help in your detective work.

(a)
```
m = 200; Z = randn(m,m);
A = Z'*Z; b = randn(m,1);
tic; x = A\b; toc;
       elapsed_time = 1.0368
```

(b)
```
tic; x = A\b; toc;
       elapsed_time = 1.0303
```

(c)
```
A2 = A; A2(m,1) = A2(m,1)/2;
tic; x = A2\b; toc;
       elapsed_time = 2.0361
```

(d)
```
I = eye(m,m); emin = min(eig(A));
A3 = A - .9*emin*I;
tic; x = A3\b; toc;
       elapsed_time = 1.0362
```

(e)
```
A4 = A - 1.1*emin*I;
tic; x = A4\b; toc;
       elapsed_time = 2.9624
```

(f)
```
A5 = triu(A);
tic; x = A5\b; toc;
       elapsed_time = 0.1261
```

(g)
```
A6 = A5; A6(m,1) = A5(1,m);
tic; x = A6\b; toc;
       elapsed_time = 2.0012
```